PROJECT REQUIREMENT DOCUMENT – CSCI 5410/W26

Title: WinterCloud Dispatch System (WCDS)
Team Size: ~3 Members
Project Length: Approx. 2.5 months
Project Leader: Prashant Rawat

Objective:

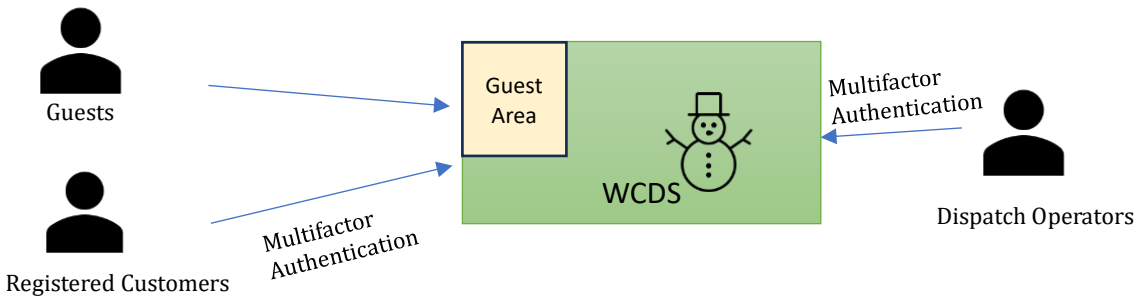
The primary objective is to build a cloud data plumbing system designed to manage and automate winter service requests (e.g., snow removal, de-icing). The application must use serverless technologies to process on-demand data and should be easy for various client types to use. You will be building a cloud data plumbing system utilizing various backend services and a simple front-end application.

Plagiarism Policy:

- This project is a group task. Collaboration of any type (outside the assigned group) amounts to a violation of the academic integrity policy and will be reported to the AIO.
- Content cannot be copied verbatim from any source(s). Please understand the concept and write in your own words. In addition, cite the actual source. Failing to do so will be considered as plagiarism and/or cheating.
- AI (Artificial Intelligence) tools cannot be used for any part of the project component. AI policy mentioned in syllabus must be followed.
- The Dalhousie Academic Integrity policy applies to all material submitted as part of this course. Please understand the policy, which is available at: https://www.dal.ca/dept/university_secretariat/academic-integrity.html

Hypothetical Scenario:

WinterSoft Technologies is developing a serverless **WinterCloud Dispatch System (WCDS)** using **a multi-cloud deployment model** and backend-as-a-service (**BaaS**). The WCDS must provide customized features and tracking services for authorized clients and limited functionality for guests. The system should feature an online virtual assistant and functionality for communication between registered clients and service dispatchers.



saurabh.dey@dal.ca
This content is protected and may not be shared, uploaded, or distributed.

Use Case: WCDS User Types

There will be three types of users in the WinterCloud Dispatch System:

1. **Guests:**
   - Can visit the application site to check the availability and tariffs of different services (e.g., residential snow removal, commercial de-icing).
   - Can use the **virtual assistant/chatbot** to get basic navigation information.
   - Should be able to view overall feedback and polarity (sentiment) received for a specific service.

2. **Registered Clients (Customers):**
   - Can perform all operations available to a guest user.
   - Will receive a notification upon successful registration and successful sign-in.
   - Will access the system using **multi-factor authentication** (1st: userID/password; 2nd: Question-Answer; 3rd: solving clue using Caesar cipher). Each factor **must be validated sequentially**.
   - Once logged in, they can **reserve a service** (e.g., snow removal) for a specific date and time.
   - Can use the virtual assistant to navigate, obtain the service reference code and details based on the booking reference code, and initiate communication with a dispatcher.
   - Should be able to provide feedback using a standard form about the services rendered.

3. **Dispatch Operators (Admin Users/Franchise):**
   - Can perform all operations available to a guest user.
   - Will receive a notification upon successful registration and sign-in.
   - Will also access the system using the same sequential **multi-factor authentication** process.
   - Once logged in, they can **add/update services** (e.g., adding a new service, adjusting hourly rates, adding discount codes).
   - Can use the virtual assistant to navigate and obtain service details based on the booking reference code.
   - Should be able to **communicate with the client** once a service ticket (issue_ticket) is received. This communication **must be asynchronous**, with synchronous communication being a potential additional feature.

saurabh.dey@dal.ca
This content is protected and may not be shared, uploaded, or distributed.

TASK DETAILS

Module Details:

You must use **Serverless architecture** to build the following modules:

1. **User Management & Authentication Module:**
   Implement sign-up validation and multi-factor authentication using a light-weight front-end application and backend services.
   - User details stored in **DynamoDB** (or equivalent NoSQL database).
   - 1st factor (User ID/Password) handled by **AWS Cognito**.
   - 2nd factor (Question/Answer) handled by **DynamoDB + AWS Lambda**.
   - 3rd factor (Caesar cipher) handled by **AWS Lambda + DynamoDB**.

2. **Virtual Assistant Module (Chatbot):**
   Bots must respond to queries. (Use AWS Lex + DynamoDB + AWS Lambda) **OR** (GCP DialogueFlow + FireStore + Cloud Function)
   - Provide online assistance for application navigation.
   - Search for service access codes and duration based on booking reference code.
   - Accept customer concerns and forward them to a Dispatch Operator.

3. **Message Passing Module:**
   Used for message passing between parties (GCP pub/sub).
   - Clients submit concerns (based on booking reference code) using a **Publisher**. A subscribed function forwards the concern to a random Dispatch Operator.
   - All communication must be logged in a NoSQL database (e.g., Firestore).

4. **Notifications Module:** Use **AWS SNS and AWS SQS**.
   - Send successful registration and successful login notifications to the registered user's email.
   - Send booking confirmation and booking failure notifications. (***Note***: Ignore payment processing, assuming it is handled in-person after booking).
   - **SNS – SQS should be used between application components**, for example, the "Service Request" entering a Standard SQS queue, which triggers a Lambda function for the "Request Approval" task.

5. **Data Analysis & Visualization:**
   The output of this module is used by all user types.
   - Dispatch Operators can check total clients and login statistics via an admin page dashboard (LookerStudio potentially embedded).
   - All user types (Guests, Clients, Operators) must see client feedback displayed in a tabular format, retrieved from a database or storage service (Firestore, GCP storage).
   - The backend must **automatically analyze the feedback sentiment** using any available natural language API and present the results in the front-end.

saurabh.dey@dal.ca
This content is protected and may not be shared, uploaded, or distributed.

6. **Web Application Building and Deployment:**
   - Build a front-end application using a suitable framework (e.g., React) to call all backend services.
   - Hosting of the application and client-facing interface must be done using **GCP CloudRun or AWS Fargate**
   - Use **CloudFormation or GCP Cloud Deployment Manager** for automated service deployment.

General Tasks:

Testing Module (GCP + AWS): Test cases must be validated. A table capturing test cases and results is required.

- Check for errors and border line cases.
- Required tests include - registered users' successful authentication, guest users' failed login, data accuracy/integrity in database upload/retrieval, chatbot response test for valid/invalid utterances, API testing, and Lambda/Cloud Function testing using test events and inspecting logs.
- Evidence of testing must be provided via screenshots.

Maintain code quality: focus on readability, reusability, cleaning, and commenting.
Perform regular **git commits**, expected mainly during Sprint 2 and Sprint 3.
Use a standard project management tool (e.g., to track status and timeline).

Deliverables – Project (40% of Course Grade)

Sprint 1 (25% of Project Grade): Point A and Point B will be graded.

A. Submit a report on project planning, document meeting agenda, background research related to cloud platforms/ services, and/or required programming.
B. It should be a single 5-page report submitted by each Team.
C. The report is due on **Feb 5, 2026** - with details mentioned in **point A**, and **point B**.
D. **Additional Submission required**: All team meetings (video and text logs) to be submitted separately.
E. Attend non-evaluative Q&A session with **Project Lead** between **Jan 26** and **Feb 5** and explain individual contribution and challenges.

Sprint 2 (25% of Project Grade):

A. Submit a 4-page report (minimum) by each team with details mentioned in point B.
B. Project planning, overall system architecture design, details of completed implementation of at least 2 modules with functional and/or validation testing of those modules (such as, "Registration", "Virtual Assistant"). Pseudo-code, flowchart/ activity diagram of completed modules. Provide Gantt chart for all other modules and next sprint.
C. The report is due on **Mar 6, 2026** - with details mentioned in point A, and point B.

saurabh.dey@dal.ca
This content is protected and may not be shared, uploaded, or distributed.

D. Additional Submission required: All team meetings (video and text logs) to be submitted separately. Flowcharts/ Architecture/ test case table etc. must be submitted separately for clarity.
E. Meeting with **Project Lead** is NOT Required for Sprint 2. However, your team can meet the Project Lead to clarify doubts or informal verification.

Sprint 3 (25% of Project):

A. Submit a 4-page report by each team with details mentioned in point B.
B. Final system architecture design, details of all pending implementations of the modules with functional and/or validation testing of those modules. Pseudo-code, flowchart/ activity diagram of all completed or partially completed modules.
C. This is the final report, which is due on **Apr 6, 2026** - with details mentioned in point A, and point B.
D. **Additional Submission required**: All team meetings (video and text logs) to be submitted separately. Flowcharts/ Architecture/ test case table etc. must be submitted separately for clarity.

Final Q&A session (10% of Project):

A. 20 min to 30 min final Q&A session. **Project Lead** and/or **Instructor** will ask questions related to the project modules and concepts.
B. Bookable Calendar will be shared for the Q&A.
C. It should be done within this timeframe **Mar 23 to Mar 31, 2026** (20 min/group)

Final Project Presentation or DEMO (15% of Project):

A. Each team needs to submit a 45 min (40 min+5min) recorded video with the following -
B. **40-min**: In this total 40-min time slot, each team member needs to explain their individual contributions with audio, video, and screenshare. The recording should clearly highlight how they achieved the task, business logic, coding structure, configurations etc. These videos do not require editing or merging.
C. **5-min:** In this video the screen needs to be captured to show different use cases of the application. Verbal explanations are not needed in this 5-min video.
D. Demo videos are due on **Mar 31, 2026**

saurabh.dey@dal.ca
This content is protected and may not be shared, uploaded, or distributed.

[Project sprint reports evaluation will be based on project rubric given below]

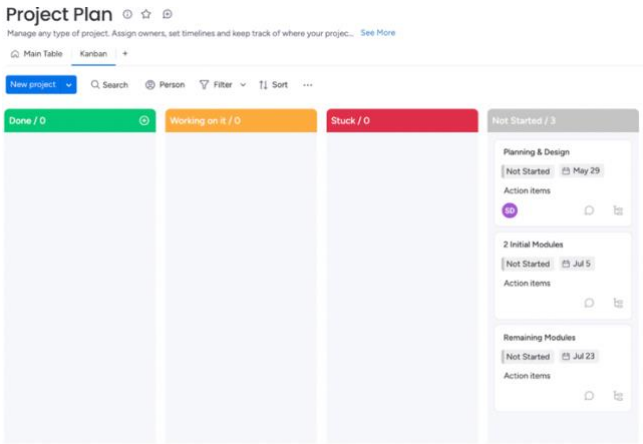
Project Report Evaluation Rubric - based on the discussion board rubric (McKinney, 2018)

	Excellent (20%)	Proficient (15%)	Marginal (10%)	Unacceptable (0%)
Completeness including Citation	All required tasks are completed	Submission highlights tasks completion. However, missed some tasks in between, which created a disconnection	Some tasks are completed, which are disjoint in nature.	Incorrect and irrelevant
Correctness	All parts of the given tasks are correct	Most of the given tasks are correct However, some portions need minor modifications	Most of the given tasks are incorrect. The submission requires major modifications.	Incorrect and unacceptable
Novelty	The submission contains novel contribution in key segments, which is a clear indication of application knowledge	The submission lacks novel contributions. There are some evidences of novelty, however, it is not significant	The submission does not contain novel contributions. However, there is an evidence of some effort	There is no novelty
Clarity	The written or graphical materials, and developed applications provide a clear picture of the concept, and highlights the clarity	The written or graphical materials, and developed applications do not show clear picture of the concept. There is room for improvement	The written or graphical materials, and developed applications fail to prove the clarity. Background knowledge is needed	Failed to prove the clarity. Need proper background knowledge to perform the tasks
Group Work	Evidence of group work, meeting logs, Coordination	Evidence of group work. However, missed meeting logs, room for improvement in Coordination	Missed meeting logs, failed to display group coordination	No group work done. Project is unacceptable

Note: Final Q&A and Presentation or DEMO will not follow the above rubric

Project Expectations:

- Please respect every individual and follow the "Culture of Respect" section provided in the course syllabus.
- All members should contribute equally to the project.
- One or two member(s) of a team completing the project cannot be considered as a successful group project, and the entire group may receive lower score.
- All members of a team may not receive similar score. Each sprint will have individual contribution grade as well.
- Try to use some form of project management software to track your project status and timeline. E.g.



saurabh.dey@dal.ca
This content is protected and may not be shared, uploaded, or distributed.